CSS4003 — DevOps 2

Lecture 10: Docker + AWS

Nauruzbayeva Farikha · Lesson 10



Recap — Lesson 9 (Cloud Storage & Docker Fundamentals)

- Image vs container? (an image is a read-only template; a container is a running instance of it with a thin writable layer)
- Why copy `requirements.txt` before `app.py`? (so the `pip install` layer stays cached when only the code changes)
- Where did the notes survive `docker rm`? (in a named volume, which outlives any container)
- Why were the S3 keys in `.env` and not in the image? (anything in an image is readable by everyone who can pull it)

Today’s agenda

- ☐ Registries: Docker Hub vs Amazon ECR
- ☐ Pushing an image to ECR: login, tag, push, scan, clean up old images
- ☐ Tags that mean something — and the `arm64` vs `amd64` trap
- ☐ Running the container on EC2 with an IAM role, not keys
- ☐ Configuration, secrets and logs for a container in AWS
- ☐ Releasing v2 and rolling back to v1
- ☐ Beyond one VM: ECS, Fargate, App Runner, EKS
- ☐ Live demo → straight into today’s practice

The scenario

Last week the Flask app finally ran the same everywhere — on **your** laptop. Now the team wants it online. Someone suggests: "just SSH into the server, `git clone`, `pip install` and run it."

The image works on my laptop — now run that exact image in the cloud, update it safely, and roll back in seconds.

Block 1

Registries and Amazon ECR

Build once, run anywhere

- The server should **never build** the app. It should **pull** the exact image you tested.
- A **registry** is where images live between "built" and "running".
- The same image (same **digest**) runs on your laptop, in CI, on EC2 — so "it worked in testing" actually means something.

```
laptop / CI ──docker push──▶ registry ──docker pull──▶ EC2 / ECS
```

Docker Hub vs Amazon ECR

	Docker Hub	Amazon ECR
Who runs it	Docker Inc.	AWS, per account and region
Image name	`user/app:1.0`	`<account>.dkr.ecr.<region>.amazonaws.com/app:1.0`
Access control	Docker Hub accounts, tokens	**IAM** — users, roles, policies
Login	`docker login` with a password/token	`aws ecr get-login-password` (temporary)
Extras	Public images, official images	Scan on push, lifecycle policies

ECR's big advantage inside AWS: an EC2 instance or ECS task pulls with its **IAM role** — no registry password stored anywhere.

Create a repository

```
export AWS_DEFAULT_REGION=us-east-1
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
REGISTRY=$ACCOUNT.dkr.ecr.us-east-1.amazonaws.com
aws ecr create-repository --repository-name lab10-app \
  --image-scanning-configuration scanOnPush=true \
  --query repository.repositoryUri --output text
```

- One **repository** per app: it holds all the tags (versions) of that image.
- The URI it prints is the image name you'll use: `<account>.dkr.ecr.us-east-1.amazonaws.com/lab10-app`

Log Docker in to ECR

```
aws ecr get-login-password \
  | docker login --username AWS --password-stdin "$REGISTRY"
# Login Succeeded
```

- `get-login-password` turns your AWS credentials into a registry password. The username is always `AWS`.
- The token is **temporary** (12 hours) — log in again in each new session.
- `--password-stdin` keeps the password out of your shell history and `ps` output.

Tag and push

```
docker build -t lab10-app:v1 .
docker tag lab10-app:v1 "$REGISTRY/lab10-app:v1"
docker push "$REGISTRY/lab10-app:v1"
aws ecr list-images --repository-name lab10-app
```

- `docker tag` doesn't copy anything — it adds a second **name** for the same image.
- The registry part of the name tells `docker push` **where** to upload.
- Only layers the registry doesn't have yet are uploaded: the second push is much faster.

Scan on push

```
aws ecr describe-image-scan-findings --repository-name lab10-app \
  --image-id imageTag=v1 \
  --query 'imageScanFindings.findingSeverityCounts'
# {"HIGH": 1, "MEDIUM": 4, "LOW": 12}   ← example shape
```

- ECR scans the image's packages for known vulnerabilities (CVEs).
- Most findings come from the **base image** — another reason to use `slim` / `alpine` and rebuild regularly to pick up patches.
- Treat **CRITICAL/HIGH** as "fix before release", not as noise.

Lifecycle policy: old images cost money

```
{
  "rules": [{
    "rulePriority": 1,
    "description": "Keep only the 10 newest images",
    "selection": {
      "tagStatus": "any",
      "countType": "imageCountMoreThan",
      "countNumber": 10
    },
    "action": { "type": "expire" }
  }]
}
```

```
aws ecr put-lifecycle-policy --repository-name lab10-app \
  --lifecycle-policy-text file://ecr-lifecycle.json
```

Quick check

You opened a new terminal the next day and `docker push` fails with `no basic auth credentials`. What happened?

The ECR login token expired (12 hours). Run `aws ecr get-login-password | docker login …` again.

Block 2

Tags, digests and CPU architectures

Don’t deploy `latest`

- `latest` is just a tag name. It **moves** every time someone pushes without a tag.
- "Run `app:latest`" on Monday and on Friday can be two different apps — and you can’t tell which one is running.
- Tag with something that **never changes**:
  - a version: `v1`, `1.4.2`
  - the Git commit: `app:3f9c2ab` (`git rev-parse --short HEAD`)
- Rolling back = running the **previous tag** again.

Tag vs digest

```
docker image inspect --format '{{index .RepoDigests 0}}' "$REGISTRY/lab10-app:v1"
# …/lab10-app@sha256:d19eb065c92c…
docker pull "$REGISTRY/lab10-app@sha256:d19eb065c92c…"
docker inspect --format '{{.Config.Image}}' app   # which image a container runs
```

- A **tag** is a label that can be moved to another image.
- A **digest** (`sha256:…`) is the hash of the content: it can **never** point to anything else.
- Pull by digest when you need to be 100% sure what runs.

The `arm64` vs `amd64` trap

- Apple Silicon Macs build `linux/arm64` images by default.
- A `t3.micro` is an Intel/AMD `x86_64` (`linux/amd64`) machine.
- Run an arm64-only image there and the container dies at once with `exec format error`.

```
docker buildx build --platform linux/amd64 -t lab10-app:v1 --load .
docker image inspect --format '{{.Os}}/{{.Architecture}}' lab10-app:v1   # linux/amd64
```

Or choose an arm64 (Graviton, `t4g`) instance if your account allows it. Windows and Intel laptops build amd64 already.

Quick check

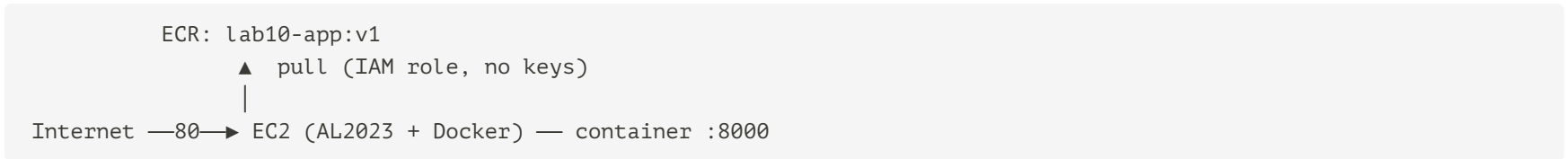
The container starts on EC2 and exits immediately. `docker logs` shows `exec /usr/local/bin/python: exec format error`. Cause?

The image was built for `arm64` on a Mac, the instance is `amd64`. Rebuild with `--platform linux/amd64` and push again.

Block 3

Running the container on EC2

The plan



- One EC2 instance, **Docker** installed by **user data** at first boot.
- The instance gets ECR access from its **instance profile** (IAM role).
- `-p 80:8000` : the internet hits port 80, the app listens on 8000.

User data: Docker on Amazon Linux 2023

```
#!/bin/bash
dnf install -y docker
systemctl enable --now docker
REGISTRY=123456789012.dkr.ecr.us-east-1.amazonaws.com
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin "$REGISTRY"
docker run -d --name app --restart unless-stopped \
  -p 80:8000 "$REGISTRY/lab10-app:v1"
```

AL2023 ships with the AWS CLI v2. User data runs once, as `root`, at the first boot. Logs: `/var/log/cloud-init-output.log`.

No keys on the server: the instance profile

- `aws ecr get-login-password` on the instance finds **no** keys in files or env… so it asks the **instance metadata service** for the role's temporary credentials.
- The role needs ECR **read** permissions (e.g. the AWS managed policy `AmazonEC2ContainerRegistryReadOnly`).
- Credentials rotate automatically; nothing to leak, nothing to copy.
- In **Learner Lab** you can't create roles; use the provided `LabInstanceProfile` (per the Learner Lab Readme).

Launching the instance

```
aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.micro --key-name vockey \
  --security-group-ids "$SG" \
  --iam-instance-profile Name=LabInstanceProfile \
  --user-data file://user-data.sh \
  --query 'Instances[0].InstanceId' --output text
```

The AMI is `x86_64` → your image must be `linux/amd64`. Security group: 80 from anywhere, 22 from your IP only (lesson 7).

Restart policies

`--restart`	Container exits by itself (crash)	You `docker stop` it	Docker / server restarts
`no` (default)	stays stopped	stays stopped	stays stopped
`on-failure`	restarted (non-zero exit)	stays stopped	restarted if it had failed
`unless-stopped`	restarted	stays stopped	restarted
`always`	restarted	stays stopped*	restarted

* with `always`, a stopped container starts again when the Docker daemon restarts. `unless-stopped` is the usual choice for a single server.

Quick check

The instance is running, but `docker pull` from ECR on it fails with `no basic auth credentials`, and there's no instance profile attached. Fix it by copying your access keys to the server?

No — attach an instance profile whose role can read ECR. Long-lived keys on a server are exactly what leaks.

Block 4

Config, secrets, logs and releases

Configuration through environment variables

```
VERSION = os.environ.get("APP_VERSION", "dev")
DATA = os.environ.get("DATA_DIR", "/data")
```

```
docker run -d --name app -p 80:8000 \
  -e APP_VERSION=v1 -e LOG_LEVEL=info \
  "$REGISTRY/lab10-app:v1"
```

- One image, many environments: dev, staging and prod differ only in **env vars**, never in code.
- Baked-in config means a **rebuild** for every small change — and secrets inside the image.

Secrets: SSM Parameter Store (or Secrets Manager)

```
aws ssm put-parameter --name /lab10/db-password \
  --type SecureString --value 'S3cr3t!'
DB_PASSWORD=$(aws ssm get-parameter --name /lab10/db-password \
  --with-decryption --query Parameter.Value --output text)
docker run -d --name app -e DB_PASSWORD="$DB_PASSWORD" \
  "$REGISTRY/lab10-app:v1"
```

- The secret lives in AWS, encrypted; the instance **reads** it at start-up through its role.
- **Secrets Manager** does the same with extras like automatic rotation.
- Concept for today: Learner Lab may not allow these services (per the Learner Lab Readme).

Logs

```
docker logs -f app           # on the instance, while it's small
docker logs --since 10m app
docker run -d --name app \
  --log-driver awslogs \
  --log-opt awslogs-region=us-east-1 --log-opt awslogs-group=/lab10/app \
  "$REGISTRY/lab10-app:v1"
```

- Apps write to **stdout**; Docker collects it.
- The `awslogs` driver sends it to **CloudWatch Logs** — logs survive the instance, and you can search them (the role needs permission to write logs).
- Concept today; CloudWatch gets its own lecture.

Releasing v2

```
# on your laptop
docker buildx build --platform linux/amd64 -t "$REGISTRY/lab10-app:v2" --push .
aws ecr list-images --repository-name lab10-app

# on the instance
docker pull "$REGISTRY/lab10-app:v2"
docker rm -f app
docker run -d --name app --restart unless-stopped -p 80:8000 "$REGISTRY/lab10-app:v2"
```

A container's image can't be changed — you **replace** the container. Data in volumes stays; everything in the old container's writable layer is gone.

Rolling back

```
docker rm -f app
docker run -d --name app --restart unless-stopped -p 80:8000 "$REGISTRY/lab10-app:v1"
curl -s localhost/version     # {"message": "...", "version": "v1"}
```

- v1 is still in ECR **and** still cached on the instance → the rollback takes seconds.
- This only works because every release has its own tag. With `latest` you'd have nothing to roll back **to**.
- Downtime: a few seconds between `rm` and `run` — acceptable here. Zero-downtime needs a load balancer and several instances (the Cloud course).

Compose on one EC2 instance

```yaml
services:
  app:
    image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/lab10-app:${TAG}
    restart: unless-stopped
    ports: ["80:8000"]
    env_file: .env
    volumes: [appdata:/data]
volumes:
  appdata:
```

```
TAG=v2 docker compose up -d     # pulls v2 and recreates only what changed
```

Fine for one server. AL2023 doesn't ship the Compose plugin — install it in user data, or move to a container service.

Quick check

You pushed a fixed image **again** as `v2` and ran `docker run …:v2` on the server. It still shows the bug. Why?

The server already had an image called `v2` cached; `docker run` doesn't re-pull an existing tag. Run `docker pull` first — or better, never reuse a tag: push `v3`.

Block 5

Beyond one VM: container services

When one VM isn’t enough

- Who restarts the container when the **instance** dies?
- Who runs 3 copies across 2 AZs, behind a load balancer?
- Who patches the OS under the containers?
- Doing all of that by hand on EC2 is exactly what **container services** automate.

Amazon ECS in three words

Concept	What it is
Task definition	"How to run my container": image, CPU/memory, ports, env, **task role**, logs
Service	"Keep N tasks running": replaces failed tasks, registers them with a load balancer
Cluster	The pool of capacity the tasks run on

The task definition is the ECS version of today's `docker run ...` line — written once, in JSON, kept in Git.

Fargate vs EC2 launch type

	Fargate	**EC2 launch type**
Servers	None to manage — AWS runs the task	Your EC2 instances in the cluster
You pay for	CPU/memory of each running task	The instances, used or not
OS patching	AWS	You
Good for	Most web apps, small teams	Special hardware, max control, steady heavy load

The rest of the family

- **App Runner** — give it an image (or repo), get an HTTPS URL; the simplest option for a web app.
- **EKS** — managed **Kubernetes**: the industry standard orchestrator, powerful and complex (DevOps courses 3+).
- **Lambda** can run container images too — for short, event-driven jobs.

Rule of thumb: start with the simplest one that fits (App Runner / ECS on Fargate); reach for EKS when you need Kubernetes itself.

Which one?

You need…	Choose
Learn and debug, one small app	Docker on one **EC2** instance (today)
A web app with HTTPS, minimal ops	**App Runner**
Several services, scaling, load balancer	**ECS on Fargate**
Kubernetes tools / portability	**EKS**
A short job triggered by an event	**Lambda** (container image)

Block 6

Security and cost

Don’t run as root

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
RUN useradd --create-home appuser \
 && mkdir -p /data && chown appuser /data
USER appuser
CMD ["python", "app.py"]
```

If someone breaks into the app, they're `appuser` inside the container, not `root`. Check with `docker exec app whoami`.

Security checklist

- **Least privilege**: the instance/task role gets ECR read, not `AdministratorAccess`.
- **No secrets** in the Dockerfile, the image, or Git — env at run time, SSM / Secrets Manager in AWS.
- **Small, pinned base images** (`python:3.12-slim`), rebuilt regularly.
- **Scan on push** and act on CRITICAL/HIGH findings.
- **Non-root** `USER` in the Dockerfile.
- **Security group**: only the ports the app needs; SSH from your IP only.

Cost hygiene

- ECR bills for **storage** of images — the lifecycle policy keeps it small.
- A running EC2 instance bills every hour, whether anyone uses the app or not.
- Clean up after the lab, in order:
  1. terminate the instance
  2. delete the security group
  3. `aws ecr delete-repository --repository-name lab10-app --force`
  4. **End Lab**

When it doesn’t work

Symptom	Likely cause
`no basic auth credentials`	Not logged in to ECR, or the 12-hour token expired
`exec format error`	arm64 image on an amd64 instance (or the reverse)
`AccessDenied` pulling on EC2	No instance profile, or its role can’t read ECR
`curl` to the public IP times out	SG has no port 80, or the container isn’t running
Old version still served	Tag reused and not re-pulled — use a new tag
Nothing happened at boot	User data failed: `/var/log/cloud-init-output.log`

Try it yourself

Track A — Learner Lab (`us-east-1`)

1. Build `linux/amd64` , push `v1` to **ECR**
2. EC2 + `LabInstanceProfile` + user data that runs the container
3. `curl` the public IP
4. Push `v2` , update the server, then **roll back** to `v1`
5. Clean up and **End Lab**

Track B — no AWS (local Docker)

1. Local registry `registry:2` on `localhost:5001` = your "ECR"
2. Tag, push, delete local copies, **pull** and run as "the server"
3. Release `v2` , roll back to `v1`
4. Compare tags and **digests**; crash the app and watch `--restart` bring it back

By the end of this lesson, you should be able to

- ☐ Explain registry, repository, tag and digest
- ☐ Push an image to Amazon ECR and keep the repository clean
- ☐ Build an image for the right CPU architecture
- ☐ Run a container on EC2 that pulls from ECR with an IAM role, not keys
- ☐ Pass configuration and secrets at run time
- ☐ Release a new version and roll back to the previous one
- ☐ Say when to use EC2 + Docker, App Runner, ECS/Fargate or EKS

Before next lecture

- ☐ Finish the practice and clean up: instance, security group, ECR repository (Track A) — or the local registry and images (Track B)
- ☐ **End Lab** in Learner Lab
- ☐ Make sure you can push to your own **GitHub** repository from the terminal

Next week the build-tag-push you did by hand today runs automatically on every push to GitHub.

Next lecture

CI/CD with GitHub Actions — build, test and push the image automatically on every commit.